Objectives

1. Validate that all implemented logic gates produce correct outputs for all input combinations.
2. Verify that the system maintains at least 60 FPS during simulation of up to 1,000 logic gates.
3. Verify that the packaging system preserves the functional behavior of circuits after compression.
4. Validate that users can successfully create and simulate a circuit within 10 minutes of first use.
5. Verify that the save project mechanism correctly generates a json file representing all component connections that can be viewed again with the load project mechanism.
6. Validate that objects can be stored in and retrieved from the inventory accurately.
7. Verify that a new project can be opened in 1 second or less after selecting “New Project”.
8. Verify that an existing project can be reopened in 4 seconds or less after selecting “Load Project”.

Prioritization

Product Capability	**Priority (High, Medium, Low)**	**Brief Justification**
Packaging	Medium	Packaging is important to the novelty of our project, and the ease of building large systems. However, our project can still function without this capability, so it shouldn’t be our highest priority.
Save/Load States	High	In order to create projects in more than one sitting a user must be able to save and reload their project reliably. If our game cannot provide this to users it creates a fundamental issue that will drive away users.
Tutorial	Low	A tutorial would be beneficial to our users, but it does not affect the functionality of our project. This should be a concern only after our project is in a functional state.
Accurate Logical Simulation	High	The core idea of our project is based around logical simulation. If our logical simulation does not work there is no point to our project, so it should be our highest priority of all.
Keyboard Game Controls	High	Standard keyboard controls for the simulation ensure that first time users are still familiar with the controls. The controls can be changed to fit the user’s needs.

Full RVTM

https://docs.google.com/spreadsheets/d/19rsaxIYxbeYNw_EG3cVFW-zsmmGv8kyXkeXLRKJI3nQ/edit?usp=sharing

Test Approach

Unit Testing

The primary focus of our unit testing are the basic circuit components. This includes the logic gates of course, but also the wires, signal propagation, saved data structures, and power source behavior. To test these components there will be automated unit tests written to cover all inputs and expected outputs (running through each gate's truth table, eg.), along with edge case testing and timing validation. All of these unit tests must pass to verify base component functionality, and then we can move forward into integration testing.

Integration Testing

Integration testing will focus on wider breadth aspects of the project. The integration testing will cover packaging, larger circuit functionality, save/load restores, inventory functionality, and frontend-backend interaction. This will be a combination of automated and manual tests. Automated testing can cover systems like packaging, timing hazards, some inventory capacity and transfers, and the testing of DLL integration of C++ logic into the unity environment. Manual testing will cover the wider integration, UI components, save/load functionality, and visual interaction with the inventory. Automated tests will be used to verify that logical accuracy holds over larger systems, that the system can save and transfer states (between the scale of whole projects or inventory states). Manual testing is to ensure that the visuals of all of these aspects clearly reflect the accuracy seen in the automated testing. Once several subsystems are able to pass their integration testing phase, system testing can begin, but it is not necessary that every aspect passes integration testing before system testing aspects.

System Testing

System testing will be where the brunt of manual testing lies. This will cover end-to-end testing of the entire system: loading, building a circuit, simulating, saving, packaging, editing, and so on. Here is where the development team will test the tutorial, visual simulation of large and packaged circuits, menu traversal, accessibility ratings, and inventory/hotbar interactions. Once these tests pass developer standards other stakeholders will be reached out to to continue system level testing along with a level of acceptance testing.

Manual Tests

Manual tests will cover UI components, UX, whole system testing, gameplay and tutorial elements, general integrated system behavior, and general exploration for undetected edge behavior.

Automated Tests

Automated testing will cover our unit tests, logical accuracy, integration for larger circuits, data storage and retrieval, timing behaviors, and regression tests.

Non-Functional Requirement Tests

Non-Functional requirements will begin during the integration testing phase but will be most relevant during the system testing phase. We will use specific measurement tools depending on the requirement, primarily quantitative, but with a mix of qualitative. Quantitative measures will include FPS

measurements for performance testing, duration measurements for speed and scalability requirements, and boolean pass/fail for logical reliability testing. Qualitative measures will most come into play for general usability, checking if other users find the system clear or confusing. For quantitative measures there already exist specific tools we have access to, but for qualitative measures we will have to create a user survey to measure how users feel about the system.

Regression Testing

Our regression testing plan will primarily rely on a github pipeline that must be compared and verified before a pull request can be approved. Once automated tests are written they can be added to the pipelined test bench, if a previously passing test fails between two pushes, a pull request will not be merged into our main branch. This will cover continued functionality of all automated tests. It should be noted that a newly generated unit test does not need to be passed for a new functionality to be pushed. If a test has never been passed it should be noted as something indicative of an issue to be fixed, but only a previously passing and now failing test should be cause for a pr to be rejected immediately. For manual tests, once save/load functionality exists we will keep a set of saved, functional circuits that can be retested with each push. This will verify both logical accuracy, and make sure the game environment stays consistent across changes.

Validation Plan

Usability and performance evaluations

- During development of the menus, inventories, and controls, it is imperative that users are able to easily navigate the game and intuit how to perform desired tasks. The user should be able to quickly find their desired settings, project pages, profile pages, block inventories, controls. If any of these processes are slow or frustrating, the user is likely to give up playing the game.
- After every milestone task of development on our Gantt chart that affects menus, inventories, or controls is completed, we will have at least three testers (not including our team) validate that they are able to easily find the following items:
    - Menu:
        - Projects folder
        - An individual project
        - How to edit a project
        - How to load a past project save
        - How to reach each button in the settings page
    - Inventory:
        - Knowing what the inventory is
        - How to open the inventory
        - Where to find each block or component in the inventory
    - Controls:
        - What each control does
- These tests will be graded with the following metrics:
    - **Task Completion Rates:** 90% of testers who are not familiar with the game are able to find the desired item in under 30 seconds.
    - **User Satisfaction:** 90% of users state that they did not experience confusion when finding the desired item, and instead, the layout of the game was intuitive.

- If any of the success metrics are not met, the prototype will be edited to include changes necessary to meet those metrics.

Pilot or prototype testing procedures

- Our primary consideration when validating the overall prototype throughout development is ensuring the user is able to create the circuits that they want to create. We want the user to be able to easily create whatever is on their mind. Unlike verification, this requires testing with external testing volunteers, because we are not just focused on if the game environment actually functions, but rather if it enables the test users to build their desired circuits.
- After every milestone task of development on our Gantt chart that affects user gameplay is completed, we will have at least three testers (not including our team) validate that they are able to create their desired circuits. The testers will play for an hour and then their tests will be graded with the following metrics:
    - **Task Completion Rates:** 90% of testers who are already familiar with the controls of the game are able to create any circuit of their choice in under an hour.
    - **Performance:** When testers simulate any circuit of their choosing, as long as it contains less than 1,000 logic gates, their game will maintain at least 60 frames per second.
    - **User Satisfaction:** 90% of users state that they were not hindered by any feature of the game when trying to design their desired circuit, and that the features of the game instead enabled them to build more effectively.
- If any of the success metrics are not met, the prototype will be edited to include changes necessary to meet those metrics.

Tools, Environments & Test Data Sets

Testing Tools

- Unity Test Framework: This allows for automated tests to be run in Unity, focusing on both Edit and Play modes. These test results will then be stored in XML files and used to guarantee correct functionality before a push to the main.
- GoogleTest: GoogleTest will be used to create automated tests for the C++ logic engine in the backend. These test results will also be recorded and sent in XML files.
- GitHub Actions: This is the main automation for the testing, as it will run all the automated test whenever code is pushed. GitHub Actions will check out the code whenever it is pushed and test the logic engine via GoogleTest and get the results. It will also build the Unity project and run the tests via UTF. If any of the tests fail then the code cannot be pulled.

Simulation Environments

- Unity: Unity will be the main testing environment for most of the project and will use different scenes to focus on testing of different aspects of the game.
- GitHub Runners: This is the temporary server hosted by GitHub that will run the automated tests that have been configured in GitHub Actions.

Test Data Sets

- Truth Tables: There will be a list of all the main gates truth tables and these will be checked in GoogleTest with the logic engine.

- Benchmark Circuits: Using a series of circuits whose behavior has already been verified and documented, these circuits will be run through and tested to ensure that our simulation has correct performance.

Version Control Management

- Configuration: GitHub is used to control the versions of code and to track the history. Since the automated testing is set up via GitHub Actions, the pipeline is set up to reject pushes if the tests come back negative.
- Archiving: Test logs from the automated tests and from GitHub will be saved to a file and then stored in a shared folder for the team to review.
- Manual Logs: Manual test logs will be kept whenever a test is done manually and whenever a new push is being implemented.